
Chapter 15 Concurrency Control



Concurrency Control

- ▶ To ensure the *isolation property* of transactions is preserved, the system must *control the interaction among the concurrent transactions*; this control is achieved through one of a variety of mechanisms called **concurrency control schemes**.
- ▶ There are a variety of concurrency-control schemes and each one has its own advantages. In practice, the most frequently used schemes are *two-phase locking* and *snapshot isolation*.



Outline

- ▶ **Lock-Based Protocols**
- ▶ **Deadlock Handling**
- ▶ **Multiple Granularity**
- ▶ **Timestamp-Based Protocols**
- ▶ **Validation-Based Protocols**
- ▶ **Multiversion Schemes**



Lock-Based Protocols

- ▶ A **lock** is a mechanism to control concurrent access to a data item.
- ▶ Data items can be locked in two *modes*:
 1. *shared (S) mode*. Data item can only be read. S-lock is requested using *lock-S* instruction.
 2. *exclusive (X) mode*. Data item can be both read as well as written. X-lock is requested using *lock-X* instruction.
- ▶ *Lock requests* are made to *concurrency-control manager*.
- ▶ Transaction can proceed only *after request is granted*.



Compatibility function

- ▶ We can define a *compatibility function* as follows
 - ▶ Let A and B represent arbitrary lock modes. Suppose that a transaction T_i requests a lock of mode A on item Q on which transaction T_j ($T_i \neq T_j$) currently holds a lock of mode B . If transaction T_i can be *granted* a lock on Q *immediately*, in spite of the presence of the mode B lock, then we say mode A is *compatible* with mode B .
- ▶ *Lock-compatibility matrix comp*
 - ▶ An element $\text{comp}(A, B)$ of the matrix has the value *true* if and only if mode A is compatible with mode B .

	S	X
S	true	false
X	false	false

S: shared
X: exclusive



Lock-Based Protocols

- ▶ A transaction may be *granted a lock* on an item if the requested lock is *compatible* with locks already held on the item by other transactions.
- ▶ Any number of transactions can hold *shared locks* on an item, but if any transaction holds an *exclusive lock* on the item, no other transaction may hold any lock on the item.
- ▶ If a lock *cannot be granted*, the requesting transaction is made to *wait till all incompatible locks held by other transactions have been released*. The lock is then granted.



Lock-Based Protocols

- ▶ *Locking is not sufficient to guarantee serializability.*
- ▶ Let A and B be two accounts that are accessed by transactions T_1 and T_2 . T_1 transfers \$50 from account B to account A ; T_2 displays the total amount of money in accounts A and B .

```
 $T_1$ : lock-X( $B$ );  
    read( $B$ );  
     $B := B - 50$ ;  
    write( $B$ );  
    unlock( $B$ );  
    lock-X( $A$ );  
    read( $A$ );  
     $A := A + 50$ ;  
    write( $A$ );  
    unlock( $A$ ).
```

```
 $T_2$ : lock-S( $A$ );  
    read( $A$ );  
    unlock( $A$ );  
    lock-S( $B$ );  
    read( $B$ );  
    unlock( $B$ );  
    display( $A+B$ ).
```



Lock-Based Protocols

T_1	T_2	concurrency-control manager
lock-X(B)		
read(B)		grant-X(B, T_1)
$B := B - 50$		
write(B)		
unlock(B)		
$B = 150$		
	lock-S(A)	
	read(A)	grant-S(A, T_2)
	unlock(A)	
	lock-S(B)	
		grant-S(B, T_2)
	read(B)	
	unlock(B)	
	display($A + B$)	
	$A = 100$	
	$B = 150$	
	$A + B = 250$	
lock-X(A)		
read(A)		grant-X(A, T_1)
$A := A - 50$		
write(A)		
unlock(A)		

- Suppose that the values of accounts $A = \$100$ and $B = \$200$.
- If $T_1 \rightarrow T_2$, $T_2 = \$300$.
- In the figure, T_1 and T_2 are executed concurrently. T_2 displays \$250, which is incorrect.
- Reason:** T_1 unlocked data item B too early, as a result of which T_2 saw an inconsistent state.



Lock-Based Protocols

T_1	T_2	concurrency-control manager
lock-X(B) read(B) $B := B - 50$ write(B) unlock(B)		grant-X(B, T_1)
	lock-S(A)	
	read(A)	grant-S(A, T_2)
	unlock(A)	
	lock-S(B)	
		grant-S(B, T_2)
	read(B)	
	unlock(B)	
	display($A + B$)	
		grant-X(A, T_1)
lock-X(A) read(A) $A := A - 50$ write(A) unlock(A)		

$B = 150$

$A = 100$
 $B = 150$
 $A + B = 250$

- ▶ The schedule shows the *points* at which the *concurrency control manager grants the locks*.
- ▶ The lock must be *granted in the interval of time* between the lock-request operation and the following action of the transaction. The exact moment is not important.
- ▶ The column depicting the actions of the concurrency-control manager will be omitted.

Lock-Based Protocols

- ▶ *Unlocking is delayed to the end of the transaction.*
- ▶ The sequence of reads and writes in previous schedule, which lead to an incorrect total of \$250, is no longer possible with T_3 and T_4 .

- Transaction T_3 corresponds to T_1 with **unlocking delayed**
- Transaction T_4 corresponds to T_2 with **unlocking delayed**

```
T3: lock-X(B);  
    read(B);  
    B := B - 50;  
    write(B);  
    unlock(B);  
    lock-X(A);  
    read(A);  
    A := A + 50;  
    write(A);  
    unlock(B);  
    unlock(A).
```

```
T4: lock-S(A);  
    read(A);  
    unlock(A);  
    lock-S(B);  
    read(B);  
    unlock(B);  
    display(A+B).  
    unlock(A);  
    unlock(B);
```



Deadlock

Partial Schedule 2

T_3	T_4
lock-x (B) read (B) $B := B - 50$ write (B)	
	lock-s (A) read (A)
	lock-s (B)
lock-x (A)	

- ▶ *Neither T_3 nor T_4 can make progress* — executing **lock-S(B)** causes T_4 to wait for T_3 to release its lock on B , while executing **lock-X(A)** causes T_3 to wait for T_4 to release its lock on A .

- ▶ Such a situation is called **deadlock**.
- ▶ To handle a deadlock, one of T_3 or T_4 must be *rolled back* and its locks *released*.



Deadlock (Cont.)

- ▶ If we *do not use locking*, or if we *unlock data items too soon after reading or writing them*, we may get *inconsistent states*. On the other hand, if we *do not unlock a data item before requesting a lock on another data item*, *deadlocks may occur*.
- ▶ *Deadlocks* are a necessary evil associated with locking, if we want to avoid inconsistent states.
- ▶ *Deadlocks are definitely preferable to inconsistent states*, since they can be handled by *rolling back*.
- ▶ There are ways to avoid deadlock in some situations.



Locking Protocol

- ▶ A **locking protocol** is *a set of rules followed by all transactions while requesting and releasing locks.*
- ▶ Locking protocols *restrict the set of possible schedules*, and the set of these schedules is *a proper subset of all possible serializable schedules.*
- ▶ We shall present several locking protocols that allow only conflict-serializable schedules, and thereby ensure isolation. Before doing so, we introduce some terminology.



Locking Protocol

- ▶ Let $\{T_0, T_1, \dots, T_n\}$ be a set of transactions participating in a schedule S .
- ▶ We say that T_i *precedes* T_j in S , written $T_i \rightarrow T_j$, if there exists a **data item** Q such that T_i has held lock mode A on Q , and T_j has held lock mode B on Q later, and **comp**(A, B) = **false**.
 - ▶ That precedence $T_i \rightarrow T_j$ implies that in any equivalent serial schedule, T_i must appear before T_j .
 - ▶ This graph is similar to the *precedence graph* that we used to test for *conflict serializability*. *Conflicts* between instructions correspond to *non-compatibility* of lock modes.

Locking Protocol

- ▶ *A schedule S is legal under a given locking protocol* if S is a possible schedule for a set of transactions that *follows the rules of the locking protocol*.
- ▶ Locking protocol *ensures conflict serializability if and only if all legal schedules are conflict serializable*
 $\Leftrightarrow$ *for all legal schedules the associated $\rightarrow$ relation is acyclic*.

Granting of Locks

- ▶ *Starvation* is also possible if concurrency control manager is badly designed.
- ▶ *A transaction* may be waiting for an *X-lock* on an item, while *a sequence of other transactions* request and are granted an *S-lock* on the same item.
- ▶ The transaction can *never* get the *X-lock* and is said to be *starved*.
- ▶ Concurrency control manager can be designed to prevent starvation.



Granting of Locks

- ▶ To *prevent starvation*, the concurrency-control manager grants the *lock M* to transaction T_i when
 - ▶ There is no other transaction holding a lock on Q in a mode that *conflicts* with M
 - ▶ There is no other transaction that is *waiting* for a lock on Q and that made its lock request *before* T_i
- ▶ Thus, a lock request will never get blocked by a lock request that is made later.



Two-Phase Locking Protocol

- ▶ **Two-phase locking protocol** is a protocol which *ensures conflict-serializable schedules*. It requires that each transaction issue lock and unlock requests in two phases:
 - ▶ **Phase 1: Growing Phase**
 - ▶ *transaction may obtain locks*
 - ▶ *transaction may not release locks*
 - ▶ **Phase 2: Shrinking Phase**
 - ▶ *transaction may release locks*
 - ▶ *transaction may not obtain locks*
- ▶ Initially, a transaction is in the *growing phase*. Once the transaction releases a lock, it enters the *shrinking phase*, and it can issue no more lock requests.

Two-Phase Locking Protocol - Example

- ▶ Consider previous examples, T_3 and T_4 are two phase, but T_1 and T_2 are not two phase.
- ▶ Note that the *unlock instructions do not need to appear at the end of the transaction*.
- ▶ In T_3 , we could move $\text{unlock}(B)$ instruction to just after $\text{lock-X}(A)$ instruction, and still retain the two-phase locking property.

T_1 : **lock-X(B);**

read(B);

$B := B - 50$;

write(B);

unlock(B);

lock-X(A);

read(A);

$A := A + 50$;

write(A);

unlock(A).

T_2 : **lock-S(A);**

read(A);

unlock(A);

lock-S(B);

read(B);

unlock(B);

display($A+B$).

T_3 : **lock-X(B);**

read(B);

$B := B - 50$;

write(B);

lock-X(A);

read(A);

$A := A + 50$;

write(A);

unlock(B);

unlock(A).

T_4 : **lock-S(A);**

read(A);

lock-S(B);

read(B);

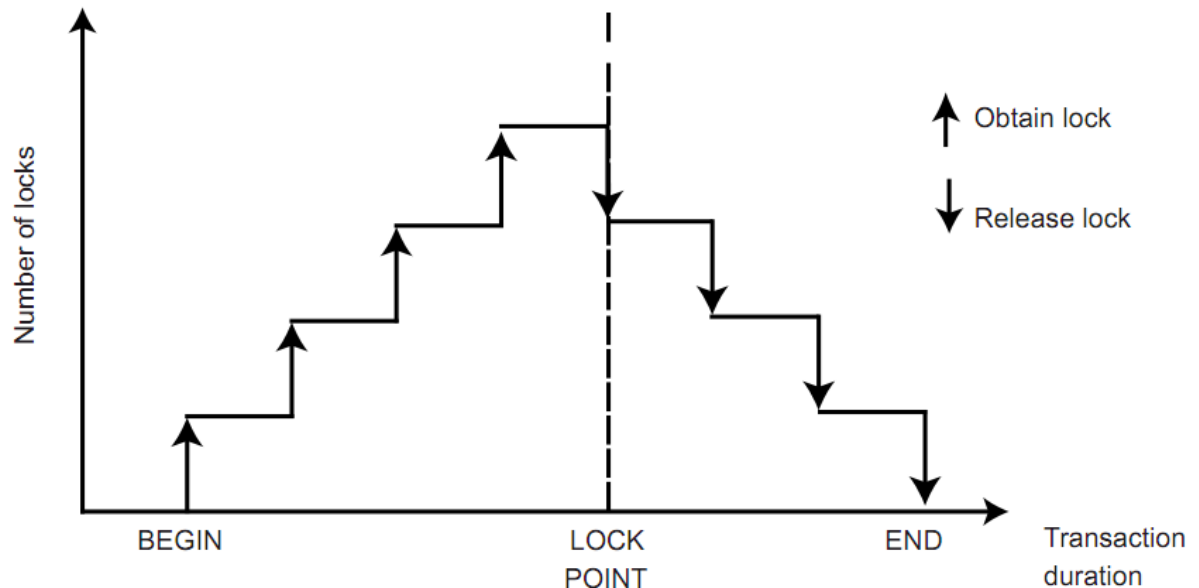
display($A+B$).

unlock(A);

unlock(B);

Two-Phase Locking Protocol

- ▶ *Two-phase locking protocol ensures conflict serializability*
 - ▶ The point in the schedule where a transaction has obtained its final lock (the end of its growing phase) is called the **lock point** of the transaction.
 - ▶ Transactions can be *ordered according to their lock points*. This ordering is a *serializability ordering* for the transactions.



Two-Phase Locking Protocol

- ▶ *Two-phase locking* does **NOT** ensure freedom from *deadlocks*
- ▶ For example, T_3 and T_4 are two phase, but they are deadlocked as shown before.

schedule 2

T_3	T_4
lock-x (B)	
read (B)	
$B := B - 50$	
write (B)	
	lock-s (A)
	read (A)
	lock-s (B)
lock-x (A)	



Two-Phase Locking Protocol

- *Cascading roll-back* is also *possible* under two-phase locking.

<i>T₅</i>	<i>T₆</i>	<i>T₇</i>
lock-X(A) read(A) lock-S(B) read(B) write(A) unlock(A)	lock-X(A) <i>read(A)</i> write(A) unlock(A)	lock-S(A) <i>read(A)</i>

- Each transaction observes the *two-phase locking protocol*
- However, the failure of *T₅* after the *read(A)* step of *T₇* leads to cascading rollback of *T₆* and *T₇*.

- *Strict two-phase locking protocol* can *avoid cascading rollback*

Two-Phase Locking Protocol

▶ **Strict two-phase locking protocol**

- ▶ A transaction must hold *all its exclusive locks* till it commits/aborts
- ▶ This requirement ensures that any data written by an *uncommitted transaction* are locked in *exclusive mode* until the transaction commits, preventing any other transaction from reading the data

▶ **Rigorous two-phase locking protocol** is even stricter:

- ▶ Here *all locks* are held till commit/abort.
- ▶ In this protocol, *transactions can be serialized in the order in which they commit.*

Lock Conversions

- ▶ Consider the following transactions T_8 and T_9 :

T_8 : read(a_1);
read(a_2);
...
read(a_n);
write(a_1);

T_9 : read(a_1);
read(a_2);
display($a_1 + a_2$);

Notice that T_8 needs an exclusive lock on a_1 only *at the end of its execution* when it writes a_1 . Thus, if T_8 could initially *lock a_1 in shared mode*, and then could later change the lock to exclusive mode, we could get more concurrency, since *T_8 and T_9 could access a_1 and a_2 simultaneously.*

- ▶ If we use two-phase locking protocol, T_8 must lock a_1 in *exclusive mode* which means *any concurrent execution of both transactions amounts to a serial execution.*

Lock Conversions

- ▶ We can provide a mechanism for *upgrading a shared lock to an exclusive lock*, and *downgrading an exclusive lock to a shared lock* (*lock conversion*)

T_8 : read(a_1);
read(a_2);
...
read(a_n);
write(a_1).

T_9 : read(a_1);
read(a_2);
display($a_1 + a_2$).

T_8	T_9
lock-S(a_1)	lock-S(a_1)
lock-S(a_2)	lock-S(a_2)
lock-S(a_3)	lock-S(a_2)
lock-S(a_4)	unlock(a_1)
lock-S(a_n)	unlock(a_2)
upgrade(a_1)	



Lock Conversions

- ▶ *Lock conversion cannot be allowed arbitrarily.*
 - ▶ *Upgrading* can take place in *only the growing phase*;
 - ▶ *Downgrading* can take place in *only the shrinking phase*.
- ▶ A transaction attempting to *upgrade* a lock on an item Q may be forced to wait. This *enforced wait* occurs if *Q is currently locked by another transaction in shared mode*.
- ▶ *Two-phase locking with lock conversion* generates *only conflict-serializable schedules* and transactions can be serialized by their *lock points*.
- ▶ If *exclusive locks* are held until the end of the transaction, the schedules are *cascadeless*.

Two-Phase Locking Protocol

- ▶ For *a set of transactions*, there may be conflict-serializable schedules that *cannot be obtained* through the *two-phase locking protocol*.
- ▶ However, to obtain *conflict-serializable schedules* through *non-two-phase locking protocols*, we need either to have *additional information* about the transactions or to impose some structure or ordering (*partial ordering*) on the *set of data items* in the database.



A Practical Scheme

- ▶ A simple but *widely used scheme* automatically generates the appropriate lock and unlock instructions on the basis of *read and write requests from the transaction*
- ▶ When a transaction T_i issues a **read(Q)** operation, the system issues a **lock-S(Q)** instruction followed by the **read(Q)** instruction.
- ▶ When T_i issues a **write(Q)** operation, the system checks to see whether T_i already holds a *shared lock* on Q . If it does, then the system issues an **upgrade(Q)** instruction, followed by the **write(Q)** instruction. Otherwise, the system issues a **lock-X(Q)** instruction, followed by the **write(Q)** instruction.
- ▶ All locks obtained by a transaction are *unlocked after that transaction commits or aborts*.

Implementation of Locking

- ▶ A *lock manager* can be implemented as *a separate process* to which transactions send lock and unlock requests.
- ▶ The lock manager replies to a *lock request* by sending *a lock grant messages* (or a message asking the transaction to *roll back*, in case of a *deadlock*).
- ▶ *Unlock messages* require only an *acknowledgment* in response, but may result in a *grant message to another waiting transaction*.

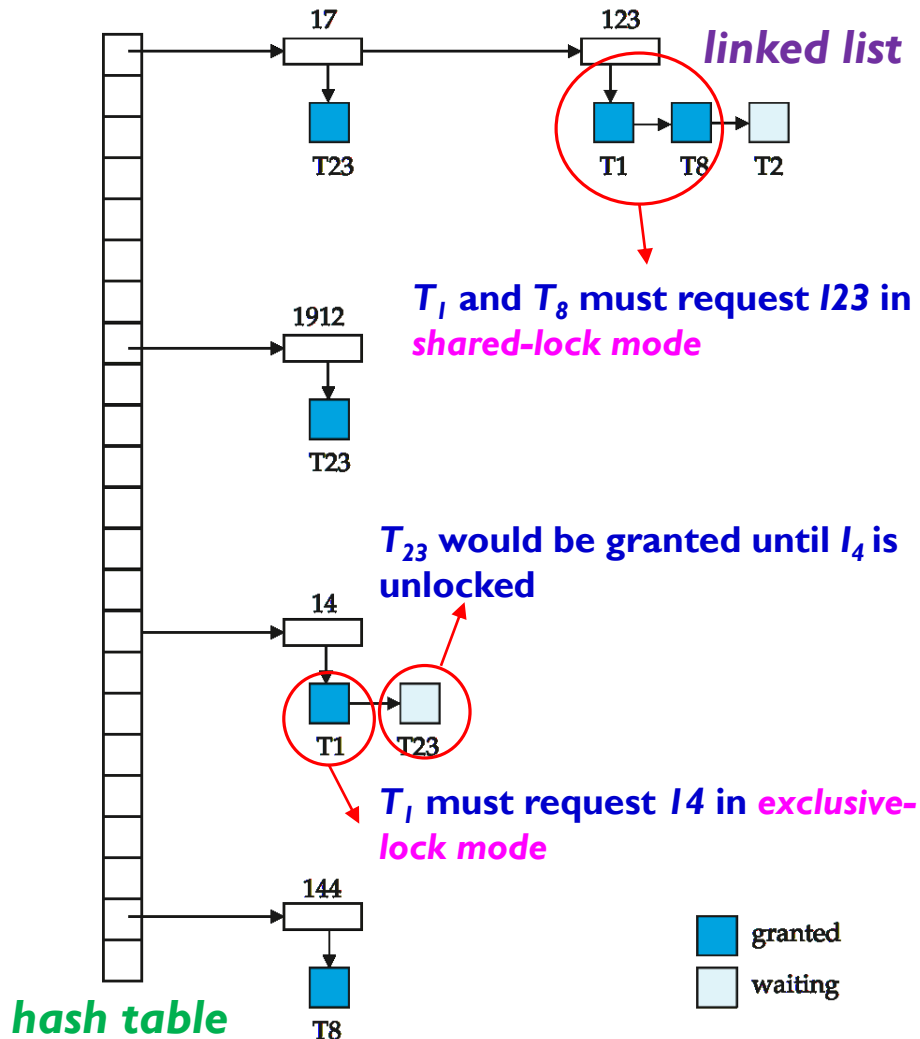


Implementation of Locking

- ▶ The lock manager maintains a data structure called a *lock table* to *record granted locks and pending requests*.
- ▶ The lock table is implemented as an *in-memory hash table* indexed on the *name of the data item being locked*.
- ▶ For each data item that is currently locked, the *lock table* maintains *a linked list of records, one for each request*, in the order in which the requests arrived. It notes:
 - ▶ *which transaction made the request*
 - ▶ *what lock mode it requested*
 - ▶ *if the request has currently been granted*



Lock Table - Example



- ▶ The table contains locks for five different data items, I₄, I₇, I₂₃, I₄₄, and I₉₁₂.
- ▶ The lock table uses *overflow chaining*, so there is a linked list of data items for each entry in the lock table.
- ▶ Dark rectangles indicate *granted locks*, and light ones indicate *waiting requests* (omitted the lock mode in this figure)



Lock Table

- ▶ The lock manager processes requests this way
 - ▶ (1) *When a lock request arrives*, it adds a record to the end of the linked list for the data item, if the linked list is present. Otherwise it creates a new linked list containing only the record for the request.
 - ▶ It *always grants a lock request on a data item that is not currently locked*.
 - ▶ If the transaction requests a lock on an item on which a lock is currently held, the lock manager grants the request *only if it is compatible with the locks that are currently held*, and *all earlier requests have been granted already*. Otherwise the request has to wait. (*No starvation*.)



Lock Table

- ▶ The lock manager processes requests this way
 - ▶ (2) *When the lock manager receives an unlock message* from a transaction, it deletes the record for that data item in the linked list corresponding to that transaction. It *tests the record that follows*, if any, to see if that request can now be granted. If it can, the lock manager grants that request, and processes the record following it, if any, similarly, and so on.
 - ▶ (3) *If a transaction aborts*, the lock manager *deletes any waiting or granted request* made by the transaction.
- ▶ The lock table should also *maintain an index on transaction identifiers*, so that it is possible to determine efficiently the set of locks held by a given transaction.

Graph-Based Protocols

- ▶ If we wish to develop protocols that are **NOT** two phase, we need *additional information* on how each transaction will access the database.
- ▶ The simplest model requires that we have *prior knowledge* about *the order in which the database items will be accessed*. Given such information, it is possible to construct locking protocols that are *not two phase*, but that, nevertheless, *ensure conflict serializability*.
- ▶ *Graph-based protocol* is such an alternative to two-phase locking.



Graph-Based Protocols

- ▶ To acquire prior knowledge, we impose a *partial ordering* $\rightarrow$ on the set $D = \{d_1, d_2, \dots, d_h\}$ of all *data items*.
- ▶ If $d_i \rightarrow d_j$ then any transaction accessing *both d_i and d_j* must *access d_i before accessing d_j* .
- ▶ The partial ordering implies that the set D may now be viewed as a *directed acyclic graph*, called a *database graph*.
- ▶ The **tree-protocol** is a simple kind of graph-based protocol.



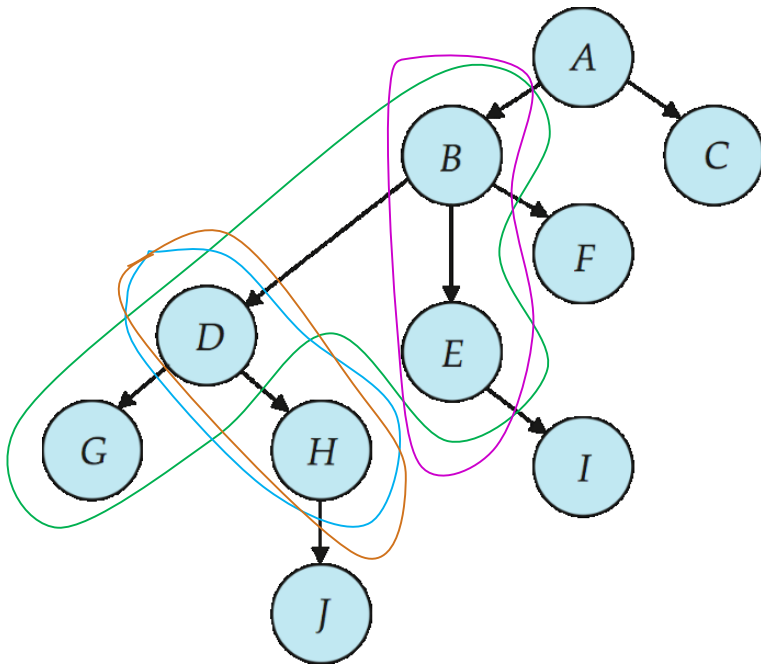
Tree Protocol

- In *tree protocol*, each transaction T_i can lock a data item *at most once*, and must observe the following rules:
1. Only *exclusive locks* are allowed.
 2. The *first lock* by T_i may be on any data item.
 3. Subsequently, a data Q can be locked by T_i *only if the parent* of Q is currently locked by T_i .
 4. Data items may be *unlocked* at any time.
 5. A data item that has been locked and unlocked by T_i cannot subsequently be *relocked* by T_i .

[Note] *The partial order ensures freedom from deadlock*

Tree Protocol - Example

- Four transactions T_{10} , T_{11} , T_{12} , T_{13} follow the tree protocol on the graph. Only lock and unlock instructions are shown.



Tree-structured database graph

One possible serializable schedule under the tree protocol

T_{10}	T_{11}	T_{12}	T_{13}
lock-X(B)	lock-X(D) lock-X(H) unlock(D)		
lock-X(E) lock-X(D) unlock(B) unlock(E)		lock-X(B) lock-X(E)	
lock-X(G) unlock(D)	unlock(H)		
		unlock(E) unlock(B)	lock-X(D) lock-X(H) unlock(D) unlock(H)
unlock(G)			

Tree Protocol

- ▶ The *tree protocol* does **NOT** ensure *recoverability* and *cascadelessness*.
- ▶ To ensure recoverability and cascadelessness, the protocol can be modified to **NOT** permit *release of exclusive locks until the end of the transaction*.
- ▶ But this may *reduce concurrency*.



Tree Protocol

- ▶ An alternative to *improve concurrency* but ensures *only recoverability*
- ▶ For each data item with an *uncommitted write*, we record which transaction performed the *last write*.
- ▶ Whenever a transaction T_i performs a *read of an uncommitted data item*, record a *commit dependency* of T_i on the transaction that performed the last write
- ▶ Transaction T_i is then not permitted to commit until the *commit of all transactions on which it has a commit dependency*. If any of these transactions *aborts*, T_i must also be aborted.



Tree Protocol

- ▶ *Advantages* of the tree protocol
 - ▶ The tree protocol ensures *conflict serializability* as well as *freedom from deadlock* (no rollbacks are required)
 - ▶ *Unlocking may occur earlier* in the tree-locking protocol than in the two-phase locking protocol (earlier unlocking may reduce waiting time and improve concurrency).

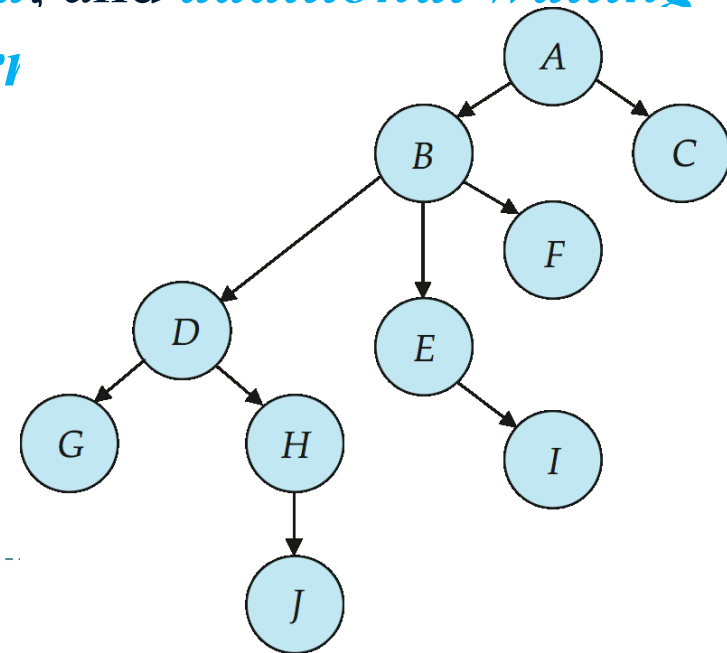


Tree Protocol (Cont.)

- ▶ **Drawbacks** of the tree protocol
 - ▶ The protocol does **NOT** guarantee *recoverability* or *cascade freedom* (need to introduce *commit dependencies* to ensure recoverability)
 - ▶ Transactions may have to *lock data items that they do not access* (increased *locking overhead*, and *additional waiting time*; *potential decrease in concurrency*)

E.g., a transaction that needs to access data items *A* and *J*, must lock not only *A* and *J*, but also data items *B*, *D*, and *H*.

[Note] Without prior knowledge,
the root has to be locked



Deadlock Handling

- ▶ Consider the following two transactions:

T_1 : write(X)
 write(Y)

T_2 : write(Y)
 write(X)

- ▶ *Schedule with deadlock*, no transactions can make progress

T_1	T_2
lock-X on A write (A)	
	lock-X on B write (B)
wait for lock-X on B	wait for lock-X on A



Deadlock Handling

- ▶ *System is deadlocked* if there is a set of transactions such that *every transaction* in the set is *waiting for another transaction* in the set.
- ▶ *Two principal methods* for dealing with deadlock
 - ▶ Use a *deadlock prevention* protocol to *ensure that the system will never enter a deadlock state*.
 - ▶ Or allow the system to enter a deadlock state, and then try to recover by using a *deadlock detection* and *deadlock recovery* scheme.
 - ▶ Note: *Prevention* is commonly used if the *probability* that the system would enter a deadlock state is *relatively high*; otherwise, detection and recovery are more efficient.

Deadlock Prevention

- ▶ *Approach I*: It ensures that *no cyclic waits can occur* by *ordering the requests for locks*, or *requiring all locks to be acquired together*.
- ▶ The simplest scheme requires that *each transaction locks all its data items before it begins execution*. Moreover, either *all are locked in one step or none are locked*.
- ▶ There are two main *disadvantages* to this protocol
 - (1) it is often *hard to predict*, before the transaction begins, what data items need to be locked;
 - (2) *data-item utilization may be very low*, since many of the data items may be locked but unused for a long time.

Deadlock Prevention

- ▶ *Approach I*: It ensures that *no cyclic waits can occur* by *ordering the requests for locks*, or *requiring all locks to be acquired together*.
- ▶ Another scheme imposes an *ordering of all data items*, and to require that *a transaction lock data items only in a sequence consistent with the ordering*. (e.g. *tree protocol*)
- ▶ A variation is to use a *total order of data items*, in conjunction with *two-phase locking*. Once a transaction has locked a particular item, it cannot request locks on items that precede that item in the ordering.



Deadlock Prevention

- ▶ **Approach I**: It ensures that *no cyclic waits can occur* by *ordering the requests for locks*, or *requiring all locks to be acquired together*.
- ▶ **Advantage**
 - ▶ (1) *Easy to implement*, as long as the set of data items accessed is known when the transaction starts execution.
 - (2) *No need to change the underlying concurrency-control system* if two-phase locking is used.



Deadlock Prevention

- ▶ *Approach II*: It is closer to *deadlock recovery*, and is to use *preemption* and *transaction rollbacks*.
 - ▶ When a transaction T_j requests a lock that transaction T_i holds, the lock granted to T_i may be *preempted* by rolling back of T_i , and granting of the lock to T_j
 - ▶ To *control the preemption*, we assign a *unique timestamp*, based on a counter or on the system clock, to each transaction when it begins
 - ▶ The system uses these timestamps only to *decide whether a transaction should wait or roll back*. *Locking* is still used for concurrency control. If a transaction is *rolled back*, it retains its *old timestamp* when restarted.

More Deadlock Prevention Strategies

- ▶ *Transaction timestamps* for deadlock prevention
 - ▶ **Wait-Die Scheme:** *non-preemptive* (**new-die & old-wait**)
 - ▶ *Older transaction* may *wait for younger one* to release data item. *Younger transactions never wait for older ones*; they are *rolled back* instead.
 - ▶ A transaction may *die several times* before acquiring needed data item.
 - ▶ Example: Transactions T_{14} , T_{15} , and T_{16} have timestamps 5, 10, and 15, respectively. If T_{14} (*old*) requests a data item held by T_{15} (*new*), then T_{14} (*old*) will wait. If T_{16} (*new*) requests a data item held by T_{15} (*old*), then T_{16} (*new*) will be rolled back.

More Deadlock Prevention Strategies

- ▶ *Transaction timestamps* for deadlock prevention
 - ▶ **Wound-Wait Scheme:** *preemptive* (**new-wait & old-die**)
 - ▶ *Older transaction wounds* (forces *rollback*) *younger transaction instead of waiting for it*. *Younger transactions may wait for older ones*.
 - ▶ Example: Transactions T_{14} , T_{15} , and T_{16} have timestamps 5, 10, and 15 respectively. If T_{14} (*old*) requests a data item held by T_{15} (*new*), then the data item will be preempted from T_{14} (*old*), and T_{15} (*new*) will be rolled back. If T_{16} (*new*) requests a data item held by T_{15} (*old*), then T_{16} (*new*) will wait.
 - ▶ *Unnecessary rollbacks* may occur for both of them.

Deadlock prevention (Cont.)

▶ *Timeout-Based Schemes*

- ▶ A transaction *waits* for a lock only for *at most a specified amount of time*.
- ▶ After that, the *wait times out* and the transaction is *rolled back*. (*deadlocks free*)
- ▶ Simple to implement and works well *if transactions are short and if long waits are likely to be due to deadlocks*.
- ▶ But difficult to determine good value of the *timeout interval*. *Starvation* is also possible. Hence, it has limited applicability.



Deadlock Detection and Recovery

- ▶ If no protocol that ensures deadlock freedom employed, a *detection and recovery scheme* must be needed. An algorithm that examines the state of the system is *invoked periodically* to determine whether a deadlock has occurred. If one has, then the system must attempt to recover from the deadlock. So the system must:
 - ▶ Maintain information about the *current allocation of data items* to transactions and *any outstanding data item requests*.
 - ▶ Provide an algorithm that uses this information to *determine whether the system has entered a deadlock state*.
 - ▶ *Recover from the deadlock* when the detection algorithm determines that a deadlock exists.

Deadlock Detection

- ▶ Deadlocks can be described as a *wait-for graph*, which consists of a pair $G = (V, E)$
 - ▶ V is a set of *vertices* which consists of all the *transactions* in the system.
 - ▶ E is a set of *edges*, and each element is *an ordered pair* $T_i \rightarrow T_j$.



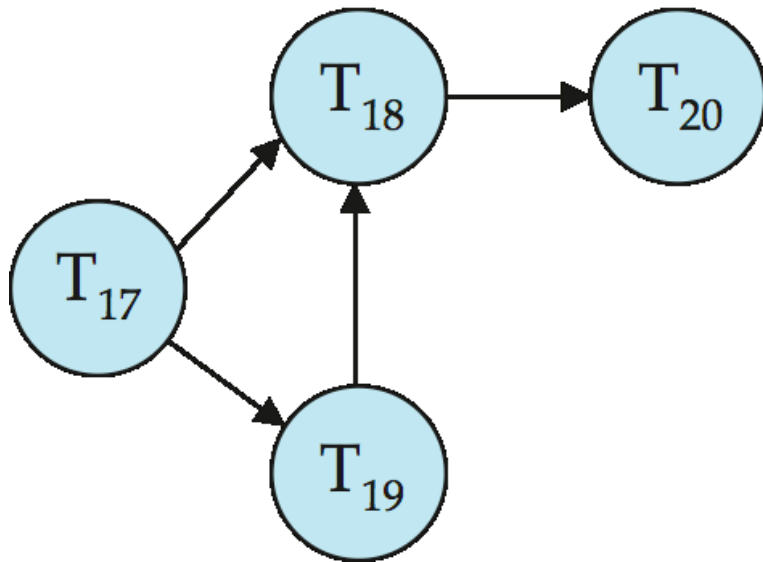
Deadlock Detection

- ▶ Deadlocks can be described as a *wait-for graph*, which consists of a pair $G = (V, E)$
 - ▶ If $T_i \rightarrow T_j$ is in E , then there is a *directed edge* from T_i to T_j , implying that T_i is *waiting for* T_j to release a data item.
 - ▶ When T_i requests a data item *currently being held* by T_j , then the edge $T_i \rightarrow T_j$ is *inserted* in the wait-for graph.
 - ▶ This edge is *removed* only when T_j is *no longer holding a data item needed by* T_i .
- ▶ The system is in a deadlock state *if and only if* the wait-for graph has a *cycle*. The system must maintain the wait-for graph and invoke a *deadlock-detection* algorithm *periodically* to look for cycles in it.

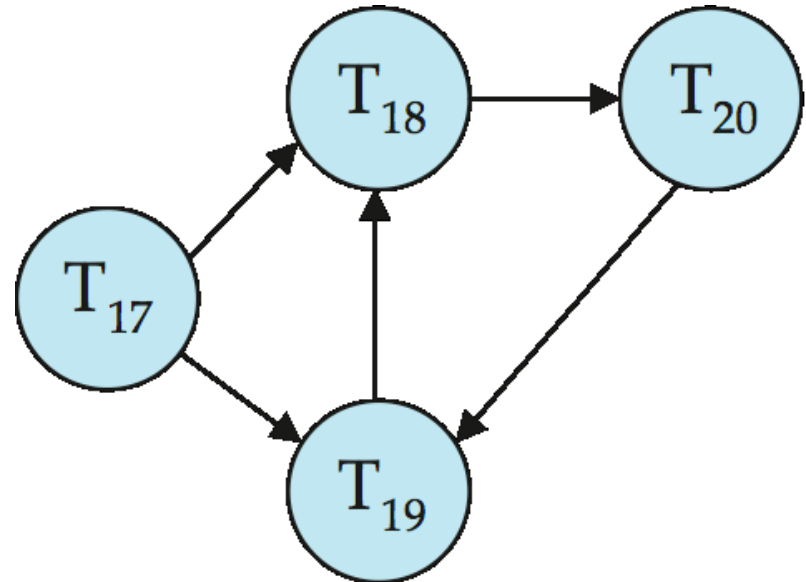


Deadlock Detection - Example

Wait-for graph without a cycle
(the system is not in a deadlock state)



Wait-for graph with a cycle
(implying that transactions T_{18} , T_{19} , and T_{20} are all deadlocked)



Deadlock Detection (Cont.)

- ▶ *When should we invoke the detection algorithm?* The answer depends on two factors:
 - ▶ *How often does a deadlock occur?*
 - ▶ *How many transactions will be affected by the deadlock?*
- ▶ If deadlocks occur *frequently*, the detection algorithm should be invoked more frequently.
- ▶ In the *worst case*, we would invoke the detection algorithm every time a request for allocation could not be granted immediately.



Deadlock Recovery

- ▶ *When deadlock is detected*
 - ▶ *One or more transactions* will have to be *rolled back* (made a victim) to break deadlock. Select that transaction as victim that will incur *minimum cost*.
 - ▶ Many factors may determine the *cost of a rollback*
 - ▶ *How long* the transaction has computed, and *how much longer* the transaction will compute before it completes its designated task.
 - ▶ *How many data items* the transaction has used.
 - ▶ *How many more data items* the transaction needs for it to complete.
 - ▶ *How many transactions* will be involved in the rollback.

Deadlock Recovery

- ▶ *When deadlock is detected*
 - ▶ *Rollback* - determine how far to roll back a transaction
 - ▶ *Total rollback*: Abort the transaction and then restart it.
 - ▶ *Partial rollback*: More effective to roll back transaction only as far as necessary to break deadlock. But it requires the system to maintain additional information about the state of all the running transactions.
 - ▶ *Starvation* happens if same transaction is always chosen as a victim. The most common solution includes the *number of rollbacks* in the cost factor to avoid starvation.



Multiple Granularity

- ▶ In the concurrency-control schemes described thus far, we have used *each individual data item* as the unit on which synchronization is performed.
- ▶ There are circumstances, however, where it would be *advantageous to group several data items*, and to treat them as one individual synchronization unit.
 - ▶ For example, if a transaction T_i needs to *access the entire database*, and a locking protocol is used, then T_i must lock each item in the database.
 - ▶ It would be better if T_i could issue a *single lock* request to lock the entire database.



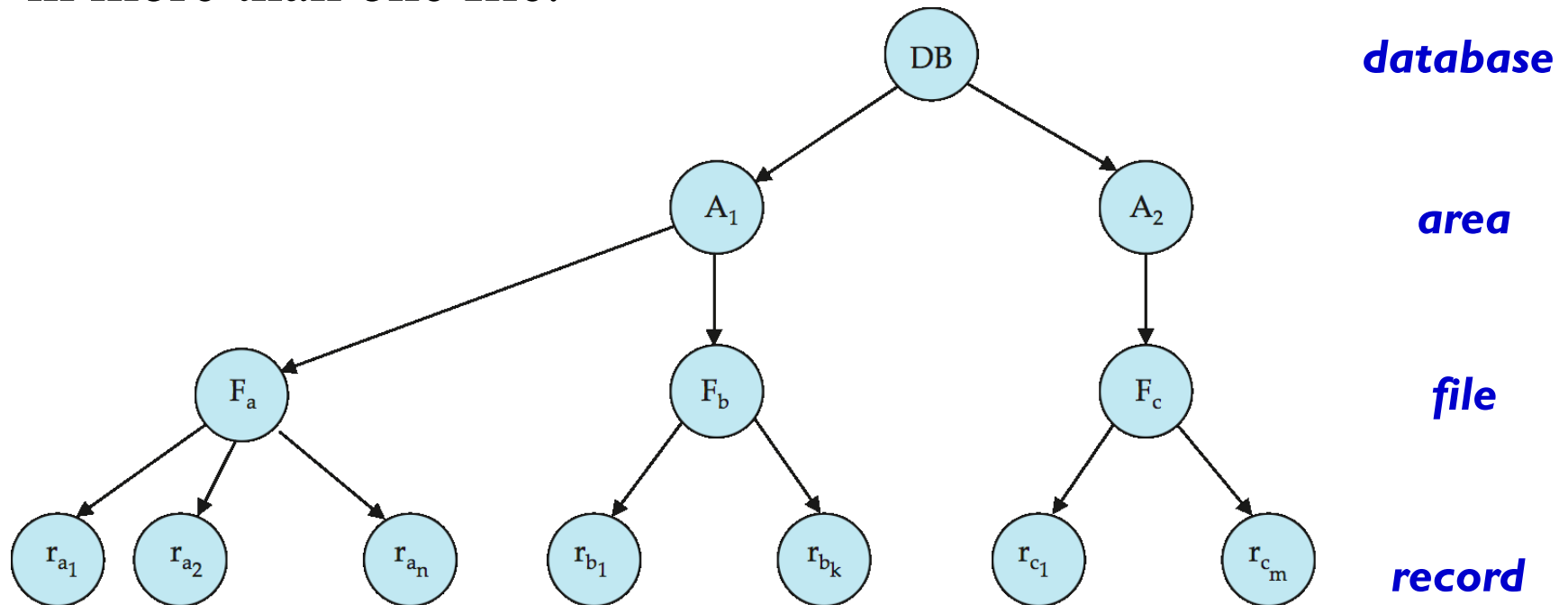
Multiple Granularity

- ▶ *Multiple granularity mechanism* allows data items to be of various sizes and define a *hierarchy of data granularities*, where the small granularities are nested within larger ones. Such a hierarchy can be represented graphically as a tree.
- ▶ Note that the tree describe here is *significantly different* from that used by the tree protocol.
 - ▶ A *non-leaf node* of the multiple-granularity tree represents the *data associated with its descendants*.
 - ▶ In the tree protocol, *each node* is an *independent data item*.



Granularity Hierarchy - Example

- ▶ The *database* consists of exactly these *areas*. Each area contains exactly those *files* that are its child nodes. No file is in more than one area. The file consists of exactly those *records* that are its child nodes, and no record can be present in more than one file.



Multiple Granularity

- ▶ Each node in the tree can be *locked individually*. When a transaction *locks a node* in the tree *explicitly*, it *implicitly locks all the node's descendants in the same mode*.
- ▶ For example, explicit locking *at the file level* involves implicit locking of *all the records in that file*.
- ▶ Suppose now that transaction T_k wishes to *lock the entire database*, it simply must *lock the root of the hierarchy*.
- ▶ To determine if the root node can be locked, one possibility is to *search the entire tree*.



Multiple Granularity

- ▶ A more efficient way is to introduce *a new class of lock modes*, called *intention lock modes*.
- ▶ If a node is locked in an *intention mode*, *explicit locking is done at a lower level of the tree* (i.e. at a finer granularity).
- ▶ *Intention locks* are put on *all the ancestors of a node* before that node is locked explicitly. Thus, a transaction *does not need to search the entire tree to determine whether it can lock a node successfully*.
- ▶ A transaction wishing to *lock a node Q*, must *traverse a path in the tree from the root to Q*. While traversing the tree, the transaction locks the various nodes in an intention mode.



Intention Lock Modes

- ▶ In addition to S and X lock modes, there are three additional lock modes with multiple granularity
 - ▶ **intention-shared (IS)**: indicates *explicit locking at a lower level of the tree* but **ONLY** with **S locks**
 - ▶ **intention-exclusive (IX)**: indicates *explicit locking at a lower level* with **exclusive OR shared locks**
- A transaction that wants to **set an S or an IS lock** on an object has to **first set IS or IX locks** on its **ancestors** (i.e., related objects of coarser granularity).
 - Similarly, a transaction that wants to **set an X or an IX lock** on an object must **set IX locks** on all of its **ancestors**.
 - **Intention locks** cannot be released on an object if the **descendants** of that object are currently locked.
- ▶ e

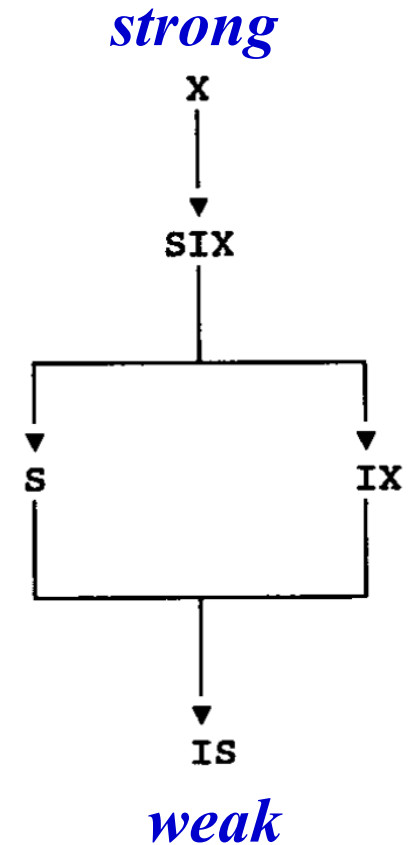
Intention Lock Modes

- ▶ In addition to S and X lock modes, there are three additional lock modes with multiple granularity
 - One additional complication arises when a transaction wants to *read an object at some granularity* and *modify some of its objects at a finer granularity*. In this case, *both an S lock and an IX lock* must be set on that object.
- ▶ *shared and intention-exclusive (SIX)*: the *subtree* rooted by that node is locked explicitly in *shared mode*, and *explicit locking* is being done at a lower level with *exclusive-mode locks*. [$SIX = S + IX$]
- ▶ Intention locks allow *a higher level node to be locked* in S or X mode *without* having to check all descendent nodes.

Compatibility Matrix

- ▶ The *compatibility matrix* for all lock modes is:

	IS	IX	S	SIX	X
IS	true	true	true	true	false
IX	true	true	false	false	false
S	true	false	true	false	false
SIX	true	false	false	false	false
X	false	false	false	false	false

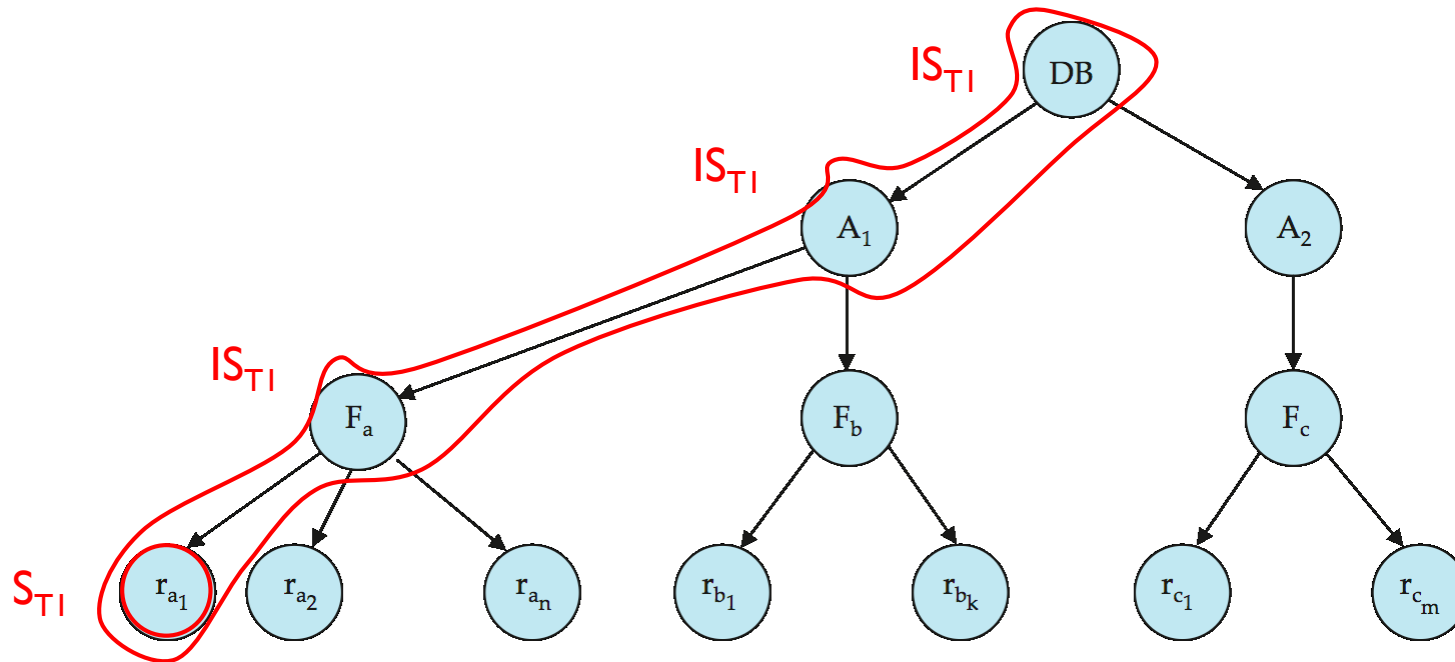


[Note] *Intention locks* are meaningful to *relations* but not to *tuples*.

Multiple Granularity Locking Protocol

- ▶ *Transaction T_i can lock a node Q* , using the following rules
 1. The *lock compatibility matrix* must be observed.
 2. The *root of tree* must be locked first and may be locked in any mode.
 3. A node Q can be locked by T_i in *S* or *IS mode* only if the parent of Q is currently locked by T_i in either IX or IS mode.
 4. A node Q can be locked by T_i in *X*, *SIX*, or *IX mode* only if the parent of Q is currently locked by T_i in either IX or SIX mode.
 5. T_i can lock a node only if it has not previously unlocked any node (that is, T_i is *two-phase*).
 6. T_i can unlock a node Q only if *none of the children* of Q are currently locked by T_i .
- ▶ Observe that locks are *acquired* in *root-to-leaf order*, whereas they are *released* in *leaf-to-root order*.

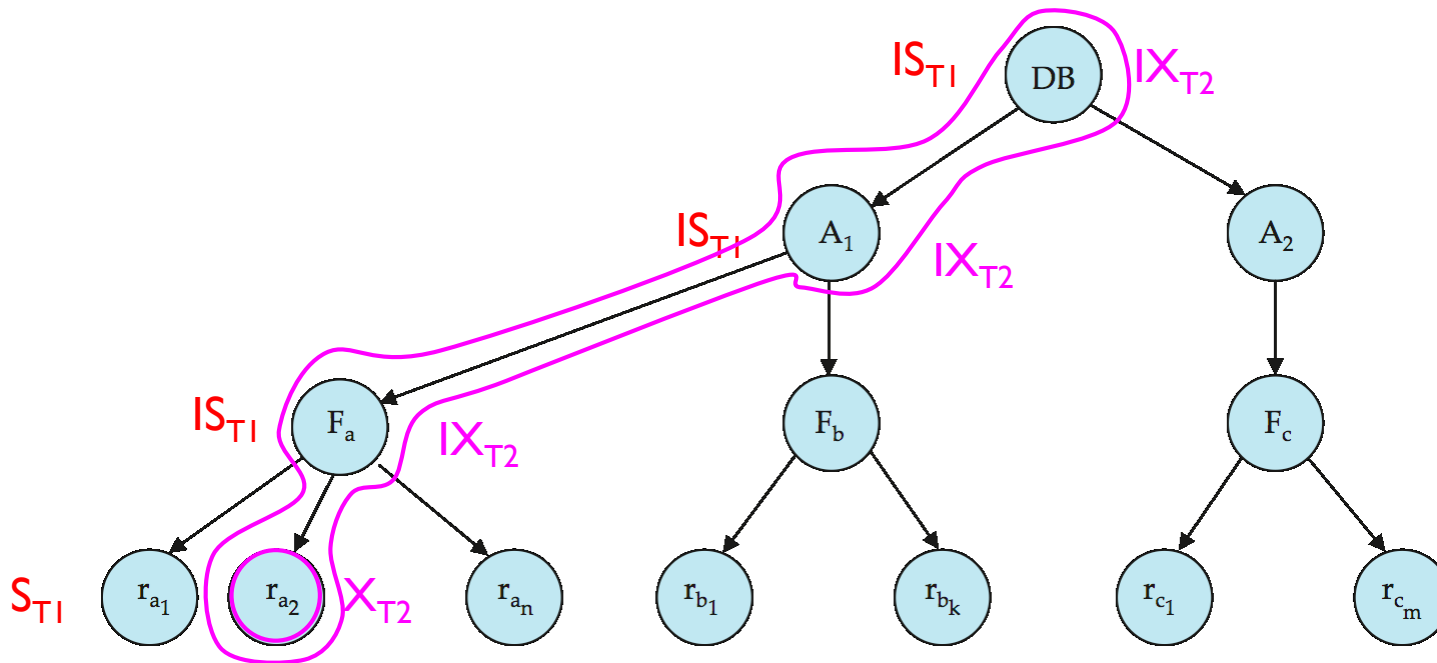
Multiple Granularity - Example



- ▶ If transaction T_1 **reads** r_{a1} , T_1 should first lock DB , A_1 , F_a in **IS mode**, and finally lock r_{a1} in **S mode**.



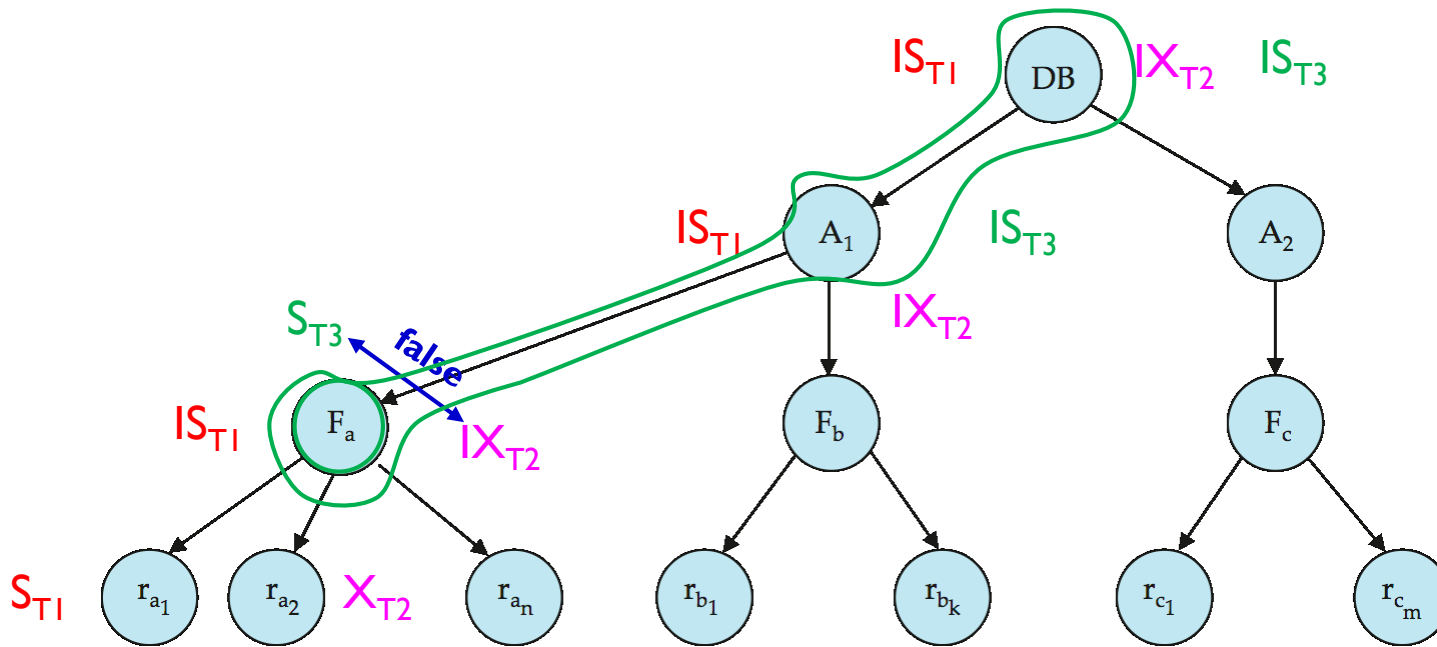
Multiple Granularity - Example



- ▶ If transaction T_2 *updates* r_{a2} , T_2 should first lock DB , A_1 , F_a in **IX mode**, and finally lock r_{a2} in **X mode**
- ▶ T_1 and T_2 can access data *concurrently*.

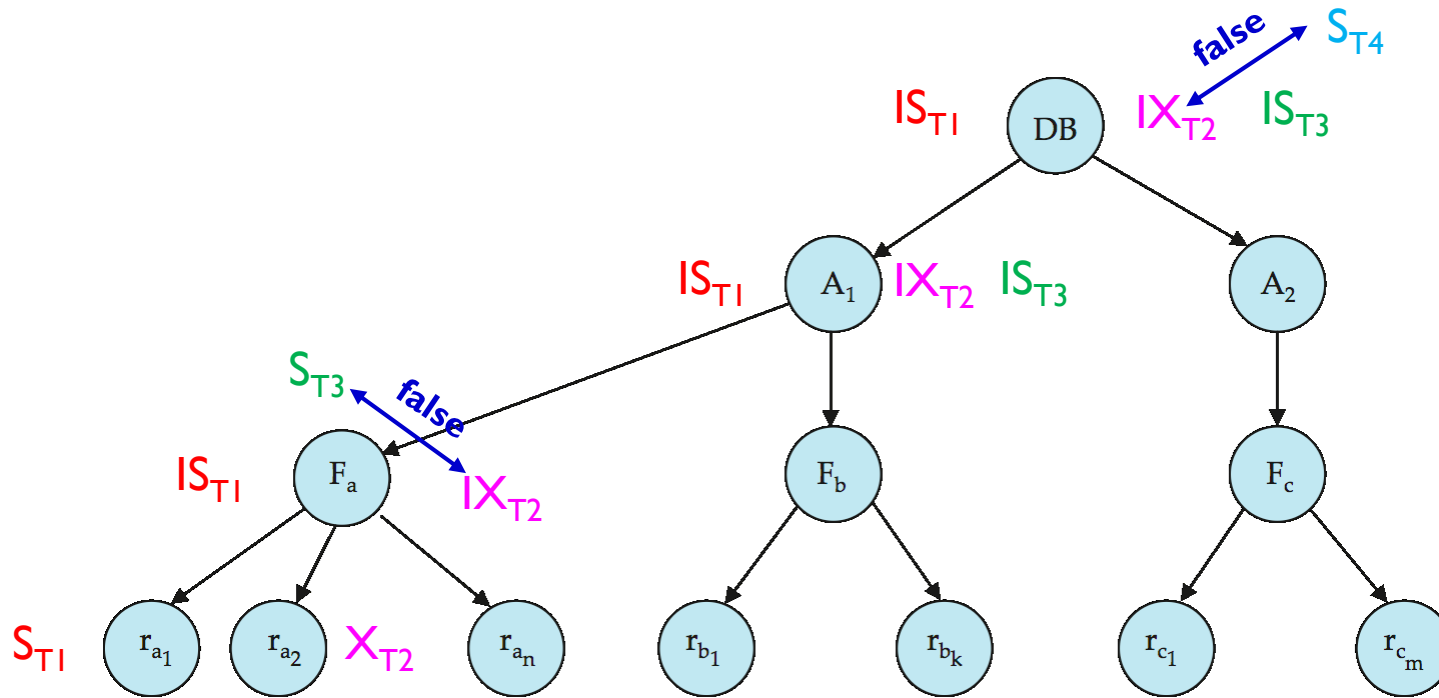


Multiple Granularity - Example



- ▶ If transaction T_3 **reads** all the records in F_a , T_3 should first lock DB , A_1 in **IS mode**, and finally lock F_a in **S mode**.
- ▶ But F_a has been locked in **IX mode**, the **S lock** request to F_a should be pended. (i.e. T_2 and T_3 **cannot** access data **concurrently**)

Multiple Granularity - Example



- ▶ If transaction T_4 **reads** all the entire database, T_4 should lock DB in **S mode**
- ▶ But DB has been locked in **IX mode**, the **S lock** request to DB should be pended.

Multiple Granularity (Cont.)

- ▶ The multiple-granularity locking protocol *enhances concurrency and reduces lock overhead*. It is particularly useful in applications that include a mix of
 - ▶ *Short transactions that access only a few data items.*
 - ▶ *Long transactions that produce reports from an entire file or set of files.*
- ▶ *Deadlock is possible* in the multiple-granularity protocol, as it is in the two-phase locking protocol.
 - ▶ There are techniques to *reduce deadlock frequency* in the multiple-granularity protocol, and also to *eliminate deadlock entirely*.



Timestamp-Based Protocols

- ▶ In the locking protocols that we have described thus far, the *order* between *every pair of conflicting transactions* is determined by the *first lock* that both members of the pair request that involves incompatible modes.
- ▶ Another method for determining the *serializability order* is to *select an ordering among transactions in advance*.
- ▶ The most common method is to use a *timestamp-ordering scheme*.



Timestamps

- ▶ With each transaction T_i , we associate a unique fixed *timestamp* $TS(T_i)$. This timestamp is assigned by the database system *before* the transaction T_i starts execution.
- ▶ If a *transaction* T_i has been assigned timestamp $TS(T_i)$, and a *new transaction* T_j enters the system, then $TS(T_i) < TS(T_j)$
- ▶ Two simple methods for *implementing TS scheme*
 - ▶ Use the value of the *system clock* as the timestamp — a transaction's timestamp is equal to *the value of the clock* when the transaction enters the system.
 - ▶ Use a *logical counter* that is incremented after a new timestamp has been assigned — a transaction's timestamp is equal to the value of the *counter* when the transaction enters the system.

Timestamps

- ▶ The timestamps *determine the serializability order*
 - ▶ If $TS(T_i) < TS(T_j)$, the produced schedule must be *equivalent to a serial schedule* in which T_i appears *before* transaction T_j
- ▶ To implement this scheme, the protocol maintains *for each data Q* two timestamp values
 - ▶ **W-timestamp(Q)** — the *largest* time-stamp of any transaction that executed **write(Q)** successfully
 - ▶ **R-timestamp(Q)** — the *largest* time-stamp of any transaction that executed **read(Q)** successfully

Timestamps are *updated* when a new instruction (**read(Q)** or **write(Q)**) is executed



Timestamp-Ordering Protocol

- ▶ The *timestamp ordering protocol* ensures that *any conflicting read and write operations are executed in timestamp order*.
- ▶ Suppose that transaction T_i issues a **read(Q)**
 1. If **$TS(T_i) < W\text{-timestamp}(Q)$** , then T_i needs to read a value of Q that was *already overwritten*. Hence, the *read operation* is *rejected*, and T_i is *rolled back*.
 2. If **$TS(T_i) \geq W\text{-timestamp}(Q)$** , then the *read operation* is executed, and $R\text{-timestamp}(Q)$ is set to *maximum* of $R\text{-timestamp}(Q)$ and $TS(T_i)$.



Timestamp-Ordering Protocol

- ▶ Suppose that transaction T_i issues **write(Q)**
 1. If **TS(T_i) < R-timestamp(Q)**, then the value of Q that T_i is producing was *needed previously*, and the system assumed that that value would *never be produced*. Hence, the **write operation** is *rejected*, and T_i is *rolled back*.
 2. If **TS(T_i) < W-timestamp(Q)**, then T_i is attempting to write an *obsolete* value of Q . Hence, this **write operation** is *rejected*, and T_i is *rolled back*.
 3. *Otherwise*, the **write operation** is executed, and W-timestamp(Q) is set to TS(T_i).

[Note] If a transaction T_i is *rolled back*, the system assigns it a *new timestamp* and restarts it.

Timestamp-based Protocols - Example

- ▶ Consider transactions T_{25} and T_{26}

T_{25} : read(B);
 read(A);
 display($A + B$).

displays the contents
of accounts A and B

T_{26} : read(B);
 $B := B - 50$;
 write(B);
 read(A);
 $A := A + 50$;
 write(A);
 display($A + B$).

transfers \$50 from account B to
account A , and then displays



Timestamp-based Protocols - Example

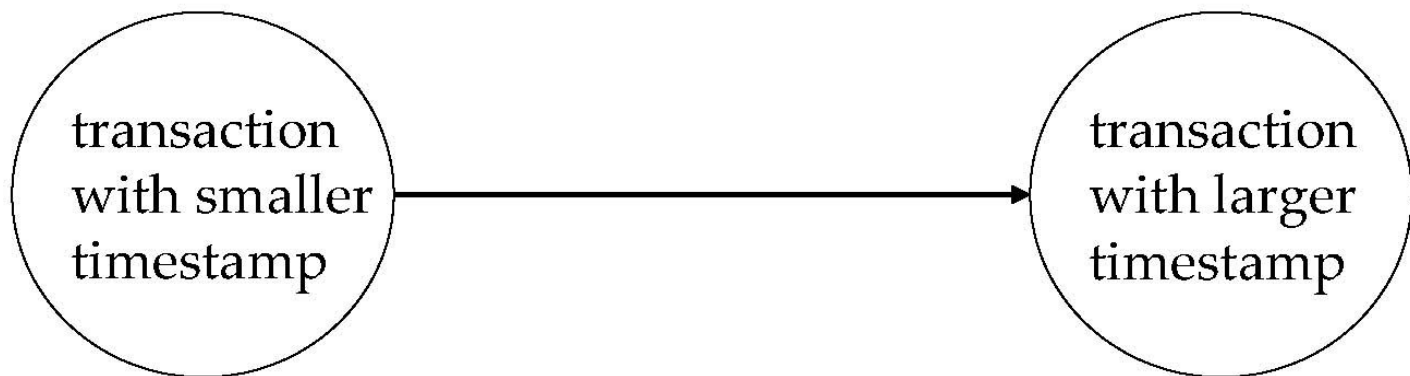
T_{25}	T_{26}
read(B)	read(B) $B := B - 50$ write(B)
read(A)	read(A)
display($A + B$)	$A := A + 50$ write(A) display($A + B$)

- ▶ Assume a transaction is assigned a timestamp immediately before its first instruction, so $\mathbf{TS}(T_{25}) < \mathbf{TS}(T_{26})$.
- ▶ It is a possible schedule under timestamp-based protocol.

[Note] (1) This schedule can also be produced by the *two-phase locking protocol*. (2) But there are schedules which are *possible* under the *two phase locking protocol*, but are *not possible* under the *timestamp protocol*, and vice versa

Timestamp-Based Protocols

- ▶ The timestamp-ordering protocol guarantees *serializability* since *all the arcs in the precedence graph are of the form*



Thus, there will be *no cycles* in the precedence graph.
Conflicting operations are processed in timestamp order.



Timestamp-Based Protocols

- ▶ Timestamp protocol *ensures freedom from deadlock as no transaction ever waits*
- ▶ However, there is a possibility of *starvation* of long transactions if *a sequence of conflicting short transactions causes repeated restarting of the long transaction.*
- ▶ If a transaction is suffering from *repeated restarts*, *conflicting transactions need to be temporarily blocked to enable the transaction to finish.*



Timestamp-Based Protocols

- ▶ *Problem with timestamp protocol*: it can generate schedules that *may not be cascade-free*, and *may not even be recoverable*.
 - ▶ Suppose T_i aborts, but T_j has read a data item written by T_i ;
 - ▶ Then T_j must abort; if T_j had been allowed to commit earlier, the schedule is not recoverable.
 - ▶ Further, any transaction that has read a data item written by T_j must abort.
 - ▶ This can lead to *cascading rollback* --- that is, a chain of rollbacks.



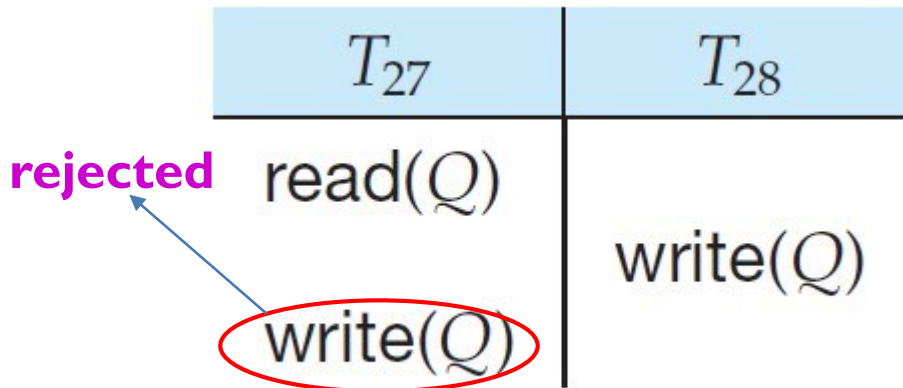
Timestamp-Based Protocols

- ▶ **Solution 1:** *Recoverability* and *cascadelessness* can be ensured by *performing all writes together at the end of the transaction* (*the writes must be atomic*: while the writes are in progress, no transaction is permitted to access any of the data items that have been written)
- ▶ **Solution 2:** *Recoverability* and *cascadelessness* can be guaranteed by using a limited form of locking, whereby *reads of uncommitted items are postponed* until the transaction that updated the item commits.
- ▶ **Solution 3:** *Recoverability alone* can be ensured by *tracking uncommitted writes*, and allowing a transaction T_i to commit only after the commit of any transaction that wrote a value that T_i read. (*commit dependencies*)



Thomas' Write Rule

- ▶ We can *allow greater concurrency* by modifying the timestamp-ordering protocol.



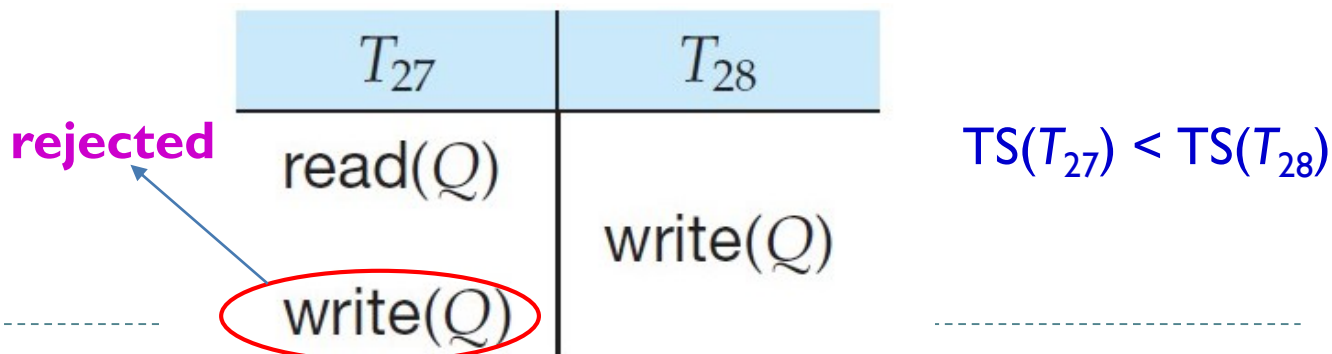
Consider the example schedule and apply the timestamp-ordering protocol

$$TS(T_{27}) < TS(T_{28})$$

- ▶ The `read( $Q$ )` operation of T_{27} *succeeds*, as does the `write( $Q$ )` operation of T_{28} .
- ▶ **W-timestamp(Q) = $TS(T_{28})$.** The `write( $Q$ )` by T_{27} is *rejected* and transaction T_{27} must be *rolled back*.

Thomas' Write Rule

- ▶ However, this rollback is *unnecessary*
 - ▶ Since T_{28} has already written Q , the value that T_{27} is attempting to write is one that will *never need to be read*
 - ▶ Any transaction T_i with $\text{TS}(T_i) < \text{TS}(T_{28})$ that attempts a $\text{read}(Q)$ will be *rolled back*, since $\text{TS}(T_i) < \text{W-timestamp}(Q)$.
 - ▶ Any transaction T_j with $\text{TS}(T_j) > \text{TS}(T_{28})$ must read the value of Q *written* by T_{28} , rather than the value that T_{27} is attempting to write.



Thomas' Write Rule

- ▶ *A modified version of the timestamp-ordering protocol: ignoring obsolete write operations*
- ▶ **Thomas' write rule**: Suppose transaction T_i issues **write(Q)**.
 1. If **TS(T_i) < R-timestamp(Q)**, then the value of Q that T_i is producing was previously needed, and it had been assumed that the value would never be produced. Hence, the system rejects the write operation and rolls T_i back. (*as before*)
 2. If **TS(T_i) < W-timestamp(Q)**, then T_i is attempting to write an obsolete value of Q . Hence, *this write operation can be ignored. (modified)*
 3. **Otherwise**, the system executes the write operation and sets W-timestamp(Q) to TS(T_i). (*as before*)

Thomas' Write Rule (Cont.)

- ▶ By ignoring the write, *Thomas' write rule* allows *schedules that are not conflict serializable but correct*.
- ▶ Those non-conflict-serializable schedules allowed satisfy the definition of *view serializable schedules*
- ▶ The following schedule is *not conflict serializable*.
However, under Thomas' write rule, the write(Q) operation of T_{27} would be ignored. The result is a schedule that is *view equivalent* to the serial schedule $\langle T_{27}, T_{28} \rangle$

T_{27}	T_{28}
read(Q)	
write(Q)	write(Q)



View Serializability

- ▶ Let S and S' be two schedules with the same set of transactions. S and S' are **view equivalent** if the following three conditions are met, for *each data item Q*

1. For each data item Q , if transaction T_i reads the **initial value** of Q in schedule S , then transaction T_i must, in schedule S' , also read the **initial value** of Q . (**read the same initial value of Q**)
2. For each data item Q , if transaction T_i executes **read(Q)** in schedule S , and if that value was produced by **a write(Q) operation executed by transaction T_j** , then the read(Q) operation of transaction T_i must, in schedule S' , also **read the value of Q that was produced by the same write(Q) operation of transaction T_j** . (**read the value of Q written by the same transaction**)
3. For each data item Q , the transaction (if any) that performs the **final write(Q)** operation in schedule S must perform the **final write(Q)** operation in schedule S' . (**perform the same final write**)

View Serializability

- ▶ A schedule S is **view serializable** if it is **view equivalent to a serial schedule**.
- ▶ **Every conflict serializable schedule is also view serializable**
- ▶ But there are **view-serializable schedules that are not conflict serializable**.

conflict serializable $\rightarrow$ **view serializable**

view serializable $\nrightarrow$ **conflict serializable**



View Serializability - Example (Cont.)

T_{27}	T_{28}	T_{29}
read(Q)	write(Q)	
write(Q)		write(Q)

blind write
(write without read first)

- ▶ This schedule is equivalent to the serial schedule $\langle T_{27}, T_{28}, T_{29} \rangle$
- ▶ In both schedules, the **read(Q)** instruction reads the *initial value of Q* , and T_{29} performs the **final write of Q** in both schedules.
- ▶ Every view serializable schedule that is not conflict serializable has *blind writes*.



Validation-Based Protocol

- ▶ If the *majority* of transactions are *read-only transactions*, the rate of conflicts among transactions may be *low*.
- ▶ Even *without the supervision of a concurrency-control scheme*, an *inconsistent state may not occur*.
- ▶ A concurrency-control scheme imposes *overhead of code execution and possible delay of transactions*. We want an alternative scheme that imposes less overhead.
- ▶ A difficulty in reducing the overhead is that we do not know in advance *which transactions will be involved in a conflict*.
- ▶ To gain that knowledge, we need a scheme for *monitoring the system*.



Validation-Based Protocol

- ▶ The **validation protocol** requires that each transaction T_i executes in two or three different phases in its lifetime, depending on whether it is a **read-only** or an **update transaction**.

1. **Read phase**. During this phase, the system executes transaction T_i . It reads the values of the various data items and stores them in variables local to T_i . It performs all write operations on **temporary local variables**, without updates of the actual database.
2. **Validation phase**. The **validation test** is applied to transaction T_i . This determines whether T_i is allowed to proceed to the write phase without causing a **violation of serializability**. If a transaction fails the validation test, the system aborts the transaction.
3. **Write phase**. If validation test **succeeds** for T_i , the temporary local variables that hold the results of any write operations performed by T_i are copied to the database. **Read-only** transactions omit this phase.

Validation-Based Protocol

- ▶ Each transaction must go through the three phases *in that order*, but phases of concurrently executing transactions can be *interleaved*.
- ▶ To perform the *validation test*, we need to know when the phases of transactions took place. Then each transaction T_i has *3 timestamps*:
 - ▶ **Start(T_i)**: the time when T_i *started* its execution
 - ▶ **Validation(T_i)**: the time when T_i *finished read phase and started its validation phase*
 - ▶ **Finish(T_i)**: the time when T_i *finished its write phase*



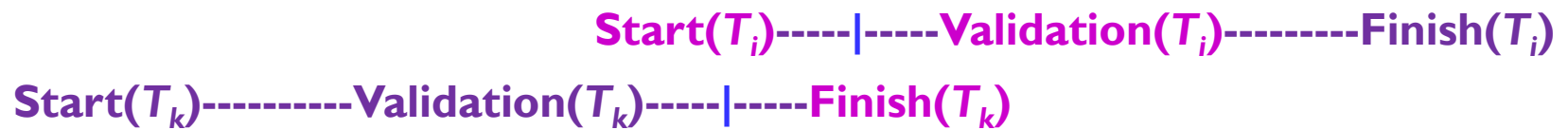
Validation-Based Protocol

- ▶ *Serializability order* is determined by *timestamp given at validation time to increase concurrency*.
- ▶ Thus $TS(T_j)$ is given the value of $Validation(T_j)$;
- ▶ And if $TS(T_j) < TS(T_k)$, then any produced schedule must be *equivalent to a serial schedule* in which T_j appears before T_k
- ▶ The reason we have chosen $Validation(T_i)$, rather than $Start(T_i)$, as the timestamp of transaction T_i is that we can expect *faster response time* provided that *conflict rates among transactions are indeed low*.



Validation-Based Protocol

- ▶ **Validation test of T_i** requires that for all T_k with $\text{TS}(T_k) < \text{TS}(T_i)$, one of the following conditions must hold:
- ▶ $\text{Finish}(T_k) < \text{Start}(T_i)$
- ▶ $\text{Start}(T_i) < \text{Finish}(T_k) < \text{Validation}(T_i)$ and the set of data items *written by T_k* does **NOT** intersect with the set of data items *read by T_i*



[Note] The second condition ensures that *the writes of T_k and T_i do not overlap*. Since the writes of T_k do not affect the read of T_i , and since T_i cannot affect the read of T_k , the *serializability order* is indeed maintained.



Schedule Produced by Validation

- ▶ Example of schedule produced using validation

T_{25}	T_{26}
read (B)	read (B) $B := B - 50$ read (A) $A := A + 50$
read (A) $\langle \text{validate} \rangle$ display ($A + B$)	$\langle \text{validate} \rangle$ write (B) write (A)

- ▶ The validation phase **succeeds** in this schedule.
- ▶ Note that the writes to the *actual variables* are performed only *after the validation phase* of T_{26} .
- ▶ Thus, T_{25} reads the *old values* of B and A , and this schedule is *serializable*.

Suppose $TS(T_{25}) < TS(T_{26})$



Validation-Based Protocol

- ▶ The *validation scheme* automatically guards against *cascading rollbacks*, since the actual writes take place only after the transaction issuing the write has committed.
- ▶ There is a possibility of *starvation* of long transactions, due to *a sequence of conflicting short transactions* that cause *repeated restarts of the long transaction*.
- ▶ To avoid starvation, conflicting transactions must be *temporarily blocked*, to enable the long transaction to finish



Validation-Based Protocol (Cont.)

- ▶ This validation scheme is also called as *optimistic concurrency control*
 - ▶ Transaction executes fully in the hope that all will go well during validation.
- ▶ In contrast, *locking and timestamp ordering* are *pessimistic*
 - ▶ They force a *wait* or a *rollback* whenever a conflict is detected, even though there is a chance that the schedule may be conflict serializable

Multiversion Schemes

- ▶ Up to this point, the concurrency-control schemes we discussed *ensure serializability* by *either delaying an operation or aborting the transaction that issued the operation*
- ▶ For example, a read operation may be *delayed* because the *appropriate value has not been written yet*; or it may be *rejected* because the *value that it was supposed to read has already been overwritten*.
- ▶ If *old copies* of each data item were kept in a system, these difficulties could be avoided



Multiversion Schemes

- ▶ ***Multiversion concurrency control*** schemes keep old versions of data item to increase concurrency.
 - ▶ *Multiversion Timestamp Ordering*
 - ▶ *Multiversion Two-Phase Locking*
- ▶ Each *successful write* results in the *creation of a new version* of the data item written.
- ▶ When a **read(Q)** operation is issued, select an *appropriate version* of Q to be read
 - ▶ *It is crucial to determine easily and quickly which version of the data item should be read*
 - ▶ *The scheme must ensure that the version to be read is selected in a manner that ensures serializability*

Multiversion Timestamp Ordering

- ▶ We can *extend the timestamp-ordering protocol to a multiversion protocol*.
- ▶ Each data item Q has *a sequence of versions* $\langle Q_1, Q_2, \dots, Q_m \rangle$, and each version Q_k contains three data fields:
 - ▶ **Content** -- the value of version Q_k
 - ▶ **W-timestamp(Q_k)** -- timestamp of the transaction that created version Q_k
 - ▶ **R-timestamp(Q_k)** -- largest timestamp of a transaction that successfully read version Q_k
- ▶ When a transaction T_i creates a new version Q_k of Q , Q_k 's **W-timestamp** and **R-timestamp** are initialized to $TS(T_i)$.
- ▶ **R-timestamp** of Q_k is *updated* whenever a transaction T_j reads Q_k , and $TS(T_j) > R\text{-timestamp}(Q_k)$.

Multiversion Timestamp Ordering

- ▶ The *multiversion timestamp-ordering scheme* operates as follows, *ensuring serializability*
- ▶ Suppose that transaction T_i issues a **read(Q)** or **write(Q)** operation. Let Q_k denote the version of Q whose **write timestamp** is the *largest write timestamp $\leq TS(T_i)$* .
 1. If transaction T_i issues a **read(Q)**, then the value returned is the content of version Q_k .
 2. If transaction T_i issues a **write(Q)**
 - ① if **$TS(T_i) < \text{R-timestamp}(Q_k)$** , then transaction T_i is *rolled back*.
 - ② if **$TS(T_i) = \text{W-timestamp}(Q_k)$** , contents of Q_k are *overwritten*
 - ③ if **$TS(T_i) > \text{R-timestamp}(Q_k)$** , a *new version* of Q is created.

Multiversion Timestamp Ordering

- ▶ We can observe that
 - ▶ *Rule 1: A transaction reads the **most recent** version that comes before it in time*
 - ▶ *Rule 2: A transaction is forced to abort if it is “**too late**” in doing a write*
- ▶ If a version is *no longer needed*, it can be *removed*
 - ▶ If Q has two versions Q_k and Q_j ($k > j$), and the oldest active transaction has timestamp $> k$ (certainly $> j$), then Q_j (*the older version*) will never be required again and can be deleted



Multiversion Timestamp Ordering

- ▶ *A desirable property* -- *a read request never fails and is never made to wait* (significant when *reading* is a more frequent operation than is writing)
- ▶ *Two undesirable properties*
 - ▶ The reading of a data item requires the updating of the *R-timestamp field*, resulting in *two potential disk accesses*, rather than one;
 - ▶ The conflicts between transactions are resolved through *rollbacks*, rather than through waits (*may be expensive*)
- ▶ This *multiversion timestamp scheme* does **NOT** ensure *recoverability* and *cascadelessness*. It can be extended in the same manner as the basic timestamp scheme.



Multiversion Two-Phase Locking

- ▶ The *multiversion two-phase locking protocol* attempts to combine the advantages of *multiversion concurrency control* with the advantages of *two-phase locking*.
- ▶ This protocol differentiates between *read-only transactions* and *update transactions*.



Multiversion Two-Phase Locking

- ▶ *Update transactions* acquire read and write locks, and hold all locks up to the end of the transaction (*update transactions follow rigorous two-phase locking*).
 - ▶ They can be *serialized according to their commit order*.
 - ▶ Each version of a data item has a single timestamp. The timestamp is *ts-counter* which is *incremented during commit*.
- ▶ *Read-only transactions* are *assigned a timestamp by reading the current value of ts-counter* before they start execution; they follow the multiversion timestamp-ordering protocol for performing reads.



Multiversion Two-Phase Locking

- ▶ When an *update transaction* wants to *read* a data item, it obtains a *shared lock* on it, and reads the *latest version*.
- ▶ When an *update transaction* wants to *write* an item, it first obtains *X lock* on it, and then *creates a new version* of the item; the write is performed on the new version, and initially sets this version's timestamp to ∞ .
- ▶ When *update transaction* T_i completes, *commit processing* occurs
 - ▶ T_i sets timestamp on the versions it has created to *ts-counter + 1*;
 - ▶ T_i increments *ts-counter* by 1



Multiversion Two-Phase Locking

- ▶ *Read-only transactions* that start **after** T_i increments *ts-counter* will see the values updated by T_i .
- ▶ *Read-only transactions* that start **before** T_i increments the *ts-counter* will see the value before the updates by T_i .
- ▶ In either case, *read-only transactions never need to wait for locks*.
- ▶ Versions are deleted in a manner like that of multiversion timestamp ordering.
- ▶ *Multiversion two-phase locking* ensures that schedules are *recoverable* and *cascadeless*.



Homework

15.2 Consider the following two transactions:

```
T34: read(A);  
      read(B);  
      if A = 0 then B := B + 1;  
      write(B).
```

```
T35: read(B);  
      read(A);  
      if B = 0 then A := A + 1;  
      write(A).
```

Add lock and unlock instructions to transactions T_{31} and T_{32} , so that they observe the two-phase locking protocol. Can the execution of these transactions result in a deadlock?



Homework

- 15.16** Consider the timestamp-ordering protocol, and two transactions, one that writes two data items p and q , and another that reads the same two data items. Give a schedule whereby the timestamp test for a write operation fails and causes the first transaction to be restarted, in turn causing a cascading abort of the other transaction. Show how this could result in starvation of both transactions. (Such a situation, where two or more processes carry out actions, but are unable to complete their task because of interaction with the other processes, is called a **livelock**.)

